Cyber Security Fraud Detection System — Complete Sequential Roadmap

1. Competition Goal

Hackathon Theme

CBI Cyber Security Hackathon 2026

Core Objective

Build an intelligent fraud detection ecosystem capable of:

- Monitoring user activity
- Tracking login patterns
- Tracking IP and location changes
- Detecting suspicious transactions
- Learning user behavior
- Generating synthetic fraud attacks
- Detecting fraud using ML models
- Notifying banks/admins in real time

---

2. Main Project Vision

The project is designed like a real banking fraud detection platform.

The system continuously:

1. Observes user activity
2. Learns normal user behavior
3. Detects anomalies
4. Generates synthetic fraud patterns
5. Competes fraud generator vs fraud detector
6. Produces fraud scores
7. Sends alerts
8. Displays everything on a live dashboard

---

3. Main Technologies Used

Technology	Purpose
Java	Backend APIs and secure transaction processing
Python	Machine Learning and fraud analysis
PostgreSQL	Database storage
React	Frontend dashboard
FastAPI	Python ML APIs
Spring Boot	Java backend framework
Docker	Deployment and local testing
JWT	Authentication
Tailwind CSS	Frontend styling

---

4. Overall System Architecture

```
Frontend Dashboard (React)
            ↓
Java Spring Boot Backend
            ↓
PostgreSQL Database
            ↓
Python ML Fraud Engine
            ↓
Fraud Detection Results
            ↓
Alert System
            ↓
Bank/Admin Notifications
```

---

5. Competition Presentation Story

Problem

Modern cyber fraud evolves rapidly. Traditional systems fail because:

- attackers change patterns
- fraud behavior evolves
- static rules become outdated

Solution

Our system:

- learns user behavior
- detects anomalies dynamically
- simulates evolving cyber attacks
- continuously improves fraud detection

This combines:

- Cyber Security
- Machine Learning
- Behavioral Analytics
- Real-Time Monitoring
- Adversarial AI

---

6. Initial Software Installation Phase

These are installed BEFORE development starts.

---

7. Install Git

Purpose

Version control and team collaboration.

Why Needed

- Track code changes
- Collaborate using GitHub

- Prevent code conflicts
- Backup project online

Install

https://git-scm.com

Verify

```
git --version
```

---

8. Install Java JDK 21

Purpose

Runs Java backend.

Why Needed

Spring Boot requires Java.

Install

https://adoptium.net

Verify

```
java -version
javac -version
```

---

9. Install Python

Purpose

Runs Machine Learning system.

Why Needed

Fraud detection models are built in Python.

Install

https://www.python.org/downloads

Verify

```
python --version
```

---

10. Install Node.js

Purpose

Runs React frontend.

Why Needed

Frontend dashboard requires Node ecosystem.

Install

https://nodejs.org

Verify

```
node -v
npm -v
```

---

11. Install PostgreSQL

Purpose

Database management.

Why Needed

Stores:

- users
- transactions
- fraud scores
- login history
- alerts

Install

https://www.postgresql.org/download

Verify

```
psql --version
```

---

12. Install Docker Desktop

Purpose

Containerized deployment.

Why Needed

Allows running:

- frontend
- backend
- ML service
- database

inside isolated containers.

Install

https://www.docker.com/products/docker-desktop

Verify

```
docker --version
```

---

13. Install VS Code

Purpose

Unified development environment.

Extensions Needed

- Java Extension Pack
- Python
- Docker
- PostgreSQL
- Tailwind CSS IntelliSense
- ESLint

Install

https://code.visualstudio.com

---

14. Install IntelliJ IDEA Community

Purpose

Professional Java development.

Why Needed

Spring Boot development becomes easier.

Install

https://www.jetbrains.com/idea

---

15. Project Folder Structure

```
project-root/
│
├── frontend-react/
├── backend-java/
├── ml-python/
├── database/
├── docker/
├── datasets/
└── docs/
```

---

16. Initialize Git Repository

Command

```
git init
```

Purpose

Tracks all project versions.

---

17. Create Environment Files

Files

```
.env
.env.example
```

Purpose

Store:

- database credentials
- JWT secrets
- API URLs
- ML service URLs

Why Important

Prevents hardcoding secrets inside code.

---

18. Database Development Phase

This is built FIRST.

Reason: Everything depends on the database structure.

---

19. Database Tables

User Tables

- users
- login_history
- user_devices
- user_locations

Transaction Tables

- transactions
- beneficiaries
- transaction_patterns

Fraud Tables

- fraud_alerts
- fraud_scores
- suspicious_sessions
- anomaly_logs

Logging Tables

- audit_logs
- notification_logs

---

20. Database Relationships

Important Relationships

```
User → Transactions
User → Login History
User → Devices
Transaction → Fraud Score
```

---

21. Why Database Comes First

Because:

- APIs depend on tables
- ML depends on stored data
- frontend depends on backend responses
- fraud analysis needs transaction history

---

22. Backend Development Phase

Now Java backend begins.

---

23. Create Spring Boot Project

Website

https://start.spring.io

Dependencies Required

Dependency	Purpose
Spring Web	REST APIs
Spring Data JPA	Database interaction
PostgreSQL Driver	PostgreSQL connection

Dependency	Purpose
Spring Security	Authentication/security
Lombok	Reduce boilerplate code
Validation	Input validation
DevTools	Auto reload

---

24. Backend Folder Structure

```
src/main/java/com/project/
│
├── config/
├── controllers/
├── services/
├── repositories/
├── models/
├── security/
├── dto/
├── middleware/
└── utils/
```

---

25. Backend Responsibilities

The backend acts as the central brain.

It:

- handles APIs
- manages authentication
- communicates with ML service
- stores transaction data
- sends alerts
- handles business logic

---

26. Authentication System

JWT Authentication

Purpose

Secure user sessions.

Flow

```
Login
  ↓
Generate JWT Token
  ↓
User sends token with requests
  ↓
Backend validates token
```

---

27. Security Features

Features

- BCrypt password hashing
- JWT authentication
- rate limiting
- IP tracking
- device fingerprinting
- session management
- audit logging

---

28. Backend APIs

Authentication APIs

```
POST /api/auth/register
POST /api/auth/login
GET /api/auth/profile
```

Transaction APIs

```
POST /api/transactions
GET /api/transactions
GET /api/transactions/:id
```

Fraud APIs

```
GET /api/fraud/alerts
GET /api/fraud/high-risk
POST /api/fraud/report
```

---

29. Why Backend Comes Before ML

Because:

- backend collects transaction data
- backend stores behavioral history
- ML models require this data
- frontend depends on backend APIs

---

30. Python ML Development Phase

Now ML service development begins.

---

31. Create Python Virtual Environment

Command

```
python -m venv venv
```

Activate

Windows

```
venv\Scripts\activate
```

Linux/macOS

```
source venv/bin/activate
```

---

32. Install ML Dependencies

Core ML Libraries

Library	Purpose
numpy	Matrix operations
pandas	Dataset processing
scikit-learn	ML algorithms
xgboost	Fraud detection model
matplotlib	Visualization
seaborn	Data plotting
joblib	Save/load models
imbalanced-learn	Handle fraud imbalance
shap	Explainable AI
fastapi	ML APIs
uvicorn	FastAPI server
psycopg2	PostgreSQL connection

---

33. Why ML is Separate from Backend

Because:

- Python is best for ML
- Java is best for scalable backend systems
- Separation improves modularity
- Easier deployment and debugging

---

34. Fraud Detection Pipeline

Step-by-Step Flow

```
Transaction Arrives
        ↓
Feature Extraction
        ↓
Behavior Analysis
        ↓
ML Fraud Prediction
        ↓
Fraud Score Generated
        ↓
Alert Triggered
```

---

35. Behavioral Features Used

Login Features

- IP address
- device fingerprint
- login timing
- browser info
- operating system
- VPN usage

Transaction Features

- amount
- frequency

- transaction timing
- beneficiary history
- merchant type

---

36. Impossible Travel Detection

Example

```
10:00 AM → Login from Delhi
10:05 AM → Login from Germany
```

Physically impossible.

Huge fraud indicator.

---

37. Fraud Detection Models

Models Used

Model	Purpose
Random Forest	Classification
XGBoost	High-performance fraud prediction
Isolation Forest	Anomaly detection

---

38. Adversarial Fraud Generator

Core Innovation

Two systems compete.

```
Fraud Generator
        VS
Fraud Detector
```

---

39. Fraud Generator Responsibilities

Generates:

- impossible travel patterns
- high-frequency transactions
- unusual login behavior
- VPN-based attacks
- midnight transaction spikes
- device switching attacks

---

40. Fraud Detector Responsibilities

Learns to detect:

- generated fraud
- real fraud
- anomalies
- behavioral deviations

---

41. FastAPI ML Service

Purpose

Expose ML model as an API.

Endpoint

```
POST /predict
```

Input transaction → returns fraud probability.

---

42. ML API Flow

```
Java Backend
      ↓
HTTP Request
```

```
      ↓
Python FastAPI
      ↓
Fraud Prediction
      ↓
Java Backend Receives Score
```

---

43. Frontend Development Phase

Now frontend development begins.

---

44. Frontend Technologies

Technology	Purpose
React	Frontend framework
Tailwind CSS	Styling
Axios	API communication
Recharts	Graphs/charts
React Router	Navigation

---

45. Frontend Dashboard Features

Dashboard Pages

- Login page
- Fraud monitoring dashboard
- Transaction history
- Risk analytics
- Device tracking
- Alert management

---

46. Live Dashboard Visualizations

Visual Components

- fraud score graphs
- login heatmaps
- suspicious transaction timeline
- user activity charts
- alert notifications

---

47. Real-Time System Interaction

Full Interaction Flow

```
User Performs Transaction
          ↓
Frontend Sends Request
          ↓
Java Backend Receives Request
          ↓
Backend Stores Transaction
          ↓
Backend Sends Data to ML Service
          ↓
ML Generates Fraud Score
          ↓
Backend Saves Fraud Score
          ↓
Alert Triggered if Risk High
          ↓
Frontend Dashboard Updates Live
```

---

48. Notification System

Features

- email alerts
- SMS alerts
- push notifications
- bank admin alerts

Tools

- Twilio
- Firebase Cloud Messaging

---

49. Explainable AI

Purpose

Explain WHY fraud was detected.

Example

```
Risk Score: 92/100
Reason:
- New Device
- Impossible Travel
- High Amount
- Unusual Time
```

---

50. Testing Phase

Backend Testing

- API testing
- JWT testing
- SQL testing

ML Testing

- false positive testing
- fraud accuracy testing
- adversarial testing

Frontend Testing

- responsiveness
- dashboard rendering
- live updates

---

51. Docker Deployment Phase

Purpose

Run entire system locally.

Containers

- frontend
- backend
- ML service
- PostgreSQL

---

52. Docker Compose

Command

```
docker-compose up
```

Purpose

Starts all services together.

---

53. Final Security Hardening

Security Additions

- HTTPS
- secure headers
- CORS restrictions
- SQL injection prevention
- XSS prevention
- request validation
- rate limiting

---

54. Final Demonstration Scenario

Demo Story

1. User logs in normally
2. Fraudulent login detected
3. Suspicious transaction generated
4. ML model flags fraud
5. Fraud score increases
6. Alert generated
7. Dashboard updates live
8. Bank/admin notified

---

55. Final Deliverables

Deliverables

- React frontend
- Spring Boot backend
- Python ML engine
- PostgreSQL database
- fraud dashboard
- real-time alerts
- documentation
- GitHub repository

---

56. Recommended Development Order

```
1. Install Software
2. Create Project Structure
3. Setup Database
4. Build Java Backend
5. Implement Authentication
6. Create APIs
7. Build Python ML Engine
8. Train Fraud Models
9. Create Fraud Generator
10. Connect Backend ↔ ML
11. Build Frontend Dashboard
12. Add Real-Time Alerts
13. Add Explainable AI
```

```
14. Test Entire System
15. Docker Deployment
16. Final Demo Preparation
```

---

57. Most Important Hackathon Advice

Do NOT overcomplicate initially.

Prioritize:

- working demo
- stable pipeline
- live dashboard
- explainability
- strong presentation
- smooth integration

Hackathons are won by:

- clarity
- execution
- practical relevance
- polished demos

more than mathematical complexity.